

Conditionals — the program that chooses

TC1037 · Algorithmic Thinking · Lecture 03

Dr. Leopoldo Rodríguez Salazar

Tuesday 29 September 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

The restaurant you love — or the new one that opened last month?

Every time you go back, you **exploit** what you know.

Every time you try the new one, you **explore**.

How do you decide?

Algorithms to Live By, ch. 2 — *Explore / Exploit*

Win-stay, lose-shift.

- If the last choice was **good** — repeat it.
- If it was **bad** — switch.

Finite · ordered · unambiguous (once you define “good”) · input: last outcome · output: tonight's choice.

Better strategies exist. Every one of them has the same skeleton: a condition, and two branches.

Your turn — 3 minutes, on paper

Write win-stay-lose-shift as an algorithm.

INPUT: OUTPUT: numbered steps.

- What do you need to *remember* from last time? Give it a name.
- How does the machine know “good”? A feeling is not a test — a number is.
- **Circle the step that is a decision.**

That circled step is what Python learns today.

The situación problema — so it is not a surprise

15% of the course. Yours: a business problem, a real dataset, a Python program.

Fri 6 Nov	Deliverable 1	the dataset (Excel, 30–50 rows, 5 text + 5 numeric columns) —
Fri 13 Nov	Deliverable 2	four to six <i>driving questions</i> — teams of 2
Fri 20 Nov	Deliverable 3	the data loaded into Python — teams of 2
Fri 27 Nov	Final	program + report + explain your code out loud — individual

- Sources: Kaggle · datos.gob.mx · World Bank · a company that lets you use its data (a family business counts).
- The final program is *functions + a menu + a main* — exactly what we build from Friday on.
- Each missed or late stage: –5 on the project. Formal release Fri 16 Oct; the full brief is on the site **now**.

Nothing to do yet — except start noticing data.

Week 1, in the margin: -- what if time = 0? --

```
1 distance_km = float(input("Distance in km: "))
2 hours = float(input("Time in hours: "))
3 speed = distance_km / hours
4 print("Average speed:", speed, "km/h")
```

X Type 0 for the time

ZeroDivisionError: float division by zero

A **correct** program, given an input it was never told how to handle.

The algorithm never said what to do when time is 0 — so the program does not know either.

A condition is a value

```
>>> 5 > 3
True
>>> hours = 0
>>> hours == 0
True
>>> type(5 > 3)
<class 'bool'>
```

< <= > >= == !=

= is a command. == is a question.

`x = 5` puts 5 in the box. `x == 5` asks whether the box holds 5 — and the answer is True or False.

if — a block that runs only when the condition is True

```
1 if hours > 0:
2     speed = distance_km / hours
3     print("Average speed:", speed, "km/h")
```

- if a condition a **colon**.
- The block: every line indented **four spaces** belongs to it.
- Un-indent and you are back in the main program — which runs *regardless*.

Move the print one level left and run with 0: `NameError: name 'speed' is not defined`.

else — the margin note, closed

```
if hours > 0:
    speed = distance_km /
        hours
    print("Average speed:",
        speed)
else:
    print("Time must be >
        0.")
```

```
1. READ distance, time
2. IF time > 0:
3.     speed <- distance /
        time
4.     DISPLAY speed
5. ELSE:
6.     DISPLAY "time must be
        > 0"
```

Exactly **one** of the two blocks runs.

The algorithm now says what happens on a bad input — and so does the program.

elif — more than two branches, in order

```
1 if grade < 0 or grade > 100:
2     print("ERROR")
3 elif grade < 70:
4     print("Reprobado")
5 elif grade < 80:
6     print("Bueno")
7 elif grade < 90:
8     print("Muy bueno")
9 else:
10    print("Excelente")
```

The first True condition wins; the rest are skipped.

Predict for 65 · 79 · 80 · 100 · 101 — then swap the < 70 and < 80 rungs and predict 65 again.

and, or, not — and the first honest use of %

A year is a leap year when it is divisible by 4 **and** not by 100, **or** when it is divisible by 400.

```
1 if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
2     print(year, "is a leap year")
3 else:
4     print(year, "is not a leap year")
```

- % is the remainder: `year % 4 == 0` means *divisible by 4*.
- Precedence: not, then and, then or. Parentheses cost nothing.
- Tests: 1984 yes · 1800 no · 2000 yes · 2026 no.

Two more shapes you will meet

The conditional expression — an if that *is a value*

```
label = "leap" if is_leap
      else "not leap"
```

Only when both branches are *values* for the same variable.

match / case (3.10+) — one value, specific alternatives

```
match option:
    case 1:
        print("Statistics")
    case 2:
        print("Chart 1")
    case _:
        print("Not an
              option")
```

The menu of the situación problema. Not for ranges.

Trinket may not have match. The if / elif ladder always works. Know both; depend on one.

Three bugs that do not crash

1. **A boundary on the wrong side.** `grade < 70` vs `grade <= 70`: a 70 fails or passes on one character.
Test 69, 70, 71 — not 50.
2. **The unreachable branch.** `if x > 10: ... elif x > 20: ...` — the second can never run. No warning.
Order rungs from most to least specific, and say so in the algorithm.
3. **A case nobody decided.** Nested “positive / negative” and 0 lands in *negative even*.
If you cannot say which branch 0 takes, you are not done.

Write the test cases before the code — one per branch, plus the boundaries.

In-class activity — speed, checked and classified · 20 min · graded

1. Read distance (km) and time (h).
2. Time not $> 0 \rightarrow$ Time must be greater than zero. — and nothing else.
3. Otherwise: compute the speed, display it, and classify it:
 < 40 slow · 40 to < 90 normal · 90 to 120 fast · > 120 too fast --- this is a road, not a runway

d	t	expected
100	0	Time must be greater than zero.
100	2	50.0 $\rightarrow$ normal
100	1	100.0 $\rightarrow$ fast
120	1	120.0 $\rightarrow$ fast
121	1	121.0 $\rightarrow$ too fast
30	1	30.0 $\rightarrow$ slow

Algorithm on paper first — IF lines written out.

Then the program, exam format. Then the tests, written under the algorithm.

Submit: photo of the algorithm + A.Matricula_ic1.py

Canvas $\rightarrow$ *In-class activity: Conditional structure (if)* — Sunday 23:59.

Rubric: algorithm 40 · six tests 40 · boundaries

Before Friday

1. **Homework 2** — leap year, three numbers descending. Due **Sunday 4 Oct, 23:59**.
2. **Lab 2, Friday 16:30** — conditionals, the same lab as every group. Laptop.
3. Levelling module: lessons 6–8 by Friday; all ten by **Thu 8 Oct**.
4. Read *Self-study: Python functions* in Canvas — 10 minutes — so `def` is not new on Friday.

Quick exam 2: Tuesday 6 October, first 20 minutes.

Conditionals and functions — read · predict · fix, plus one function to write.

The program can choose now.

Friday: it learns to keep a promise.